

User-Defined Procedures & Functions

User-Defined Procedures

2.1 What is a Procedure?

A procedure is a named PL/SQL block that performs a specific task. It is stored permanently in the database and can be called multiple times. Procedures can accept input values, return output values, or do both.

2.2 Syntax to Create a Procedure

```
CREATE [OR REPLACE] PROCEDURE procedure_name
[(parameter_name [IN | OUT | IN OUT] datatype, ...)]
IS | AS
-- Declaration section
BEGIN
-- Executable statements
EXCEPTION
-- Exception handling (optional)
END [procedure_name];
/
```

Syntax Notes

OR REPLACE → Overwrites an existing procedure with the same name.

IS / AS → Both keywords are interchangeable here.

The / → Tells SQL*Plus to execute the PL/SQL block.

2.3 Parameter Modes

Mode	Description
IN	Default mode. Passes a value INTO the procedure. Read-only inside.
OUT	Passes a value OUT from the procedure to the caller. Must be a variable.
IN OUT	Passes a value in AND receives a modified value out. Both read and write.

2.4 Creating a Simple Procedure (No Parameters)

Example: Display a welcome message.

```
CREATE OR REPLACE PROCEDURE greet_student
IS
BEGIN
  DBMS_OUTPUT.PUT_LINE('Welcome to PL/SQL Programming!');
  DBMS_OUTPUT.PUT_LINE('Oracle 10g XE - BCA Study Notes');
END greet_student;
/
```

How to Execute:

```
SET SERVEROUTPUT ON
EXECUTE greet_student;
-- OR
BEGIN
    greet_student;
END;
/
```

2.5 Procedure with IN Parameter

Example: Accept an employee ID and display their name.

```
CREATE OR REPLACE PROCEDURE display_employee (p_empno IN NUMBER)
IS
    v_ename VARCHAR2(50);
    v_sal NUMBER;
BEGIN
    SELECT ename, sal
    INTO v_ename, v_sal
    FROM emp
    WHERE empno = p_empno;

    DBMS_OUTPUT.PUT_LINE('Name : ' || v_ename);
    DBMS_OUTPUT.PUT_LINE('Salary : ' || v_sal);
EXCEPTION
    WHEN NO_DATA_FOUND THEN
        DBMS_OUTPUT.PUT_LINE('Employee not found!');
END display_employee;
/
```

Execute:

```
SET SERVEROUTPUT ON
EXECUTE display_employee(7369);
```

2.6 Procedure with OUT Parameter

Example: Retrieve salary of an employee and return it.

```
CREATE OR REPLACE PROCEDURE get_salary
(p_empno IN NUMBER, p_salary OUT NUMBER)
IS
BEGIN
    SELECT sal INTO p_salary
    FROM emp
    WHERE empno = p_empno;
EXCEPTION
    WHEN NO_DATA_FOUND THEN
        p_salary := 0;
END get_salary;
/
```

Execute with OUT parameter:

```
SET SERVEROUTPUT ON
DECLARE
    v_salary NUMBER;
BEGIN
    get_salary(7369, v_salary);
```

```
DBMS_OUTPUT.PUT_LINE('Salary is: ' || v_salary);
END;
/
```

2.7 Procedure with IN OUT Parameter

Example: Double the salary value passed.

```
CREATE OR REPLACE PROCEDURE double_salary (p_sal IN OUT NUMBER)
IS
BEGIN
    p_sal := p_sal * 2;
END double_salary;
/
```

Execute:

```
DECLARE
    v_salary NUMBER := 5000;
BEGIN
    double_salary(v_salary);
    DBMS_OUTPUT.PUT_LINE('Doubled Salary: ' || v_salary);
END;
/
```

2.8 Dropping a Procedure

```
DROP PROCEDURE procedure_name;

-- Example:
DROP PROCEDURE greet_student;
```

2.9 Viewing Procedure Source Code

```
-- View all user procedures:
SELECT object_name FROM user_objects WHERE object_type = 'PROCEDURE';

-- View source code of a procedure:
SELECT text FROM user_source WHERE name = 'GREET_STUDENT' ORDER BY line;
```

Common Mistakes to Avoid

- Forgetting SET SERVEROUTPUT ON before calling DBMS_OUTPUT.PUT_LINE.
- Using a literal value for an OUT parameter — it must be a variable.
- Forgetting the / at the end of the procedure body in SQL*Plus.
- Mismatching data types between actual and formal parameters.

Unit 3: User-Defined Functions

3.1 What is a Function?

A function is a named PL/SQL subprogram that always returns exactly one value to the calling program. Functions are used to compute and return a value. They can accept parameters (IN mode only, by default) and must include a RETURN statement.

3.2 Syntax to Create a Function

```
CREATE [OR REPLACE] FUNCTION function_name
[(parameter_name [IN] datatype, ...)]
RETURN return_datatype
IS | AS
-- Declaration section
BEGIN
-- Executable statements
RETURN expression;
EXCEPTION
-- Exception handling (optional)
END [function_name];
/
```

Important Rules for Functions

A function **MUST** have at least one RETURN statement in the BEGIN block.

A function **MUST** specify a RETURN datatype in the header.

Functions can only use IN parameters (default) — not OUT or IN OUT.

A function can be called inside a SQL SELECT statement if it is pure.

3.3 Simple Function — No Parameters

Example: Return the current system date as a string.

```
CREATE OR REPLACE FUNCTION get_today
RETURN VARCHAR2
IS
v_date VARCHAR2(30);
BEGIN
v_date := TO_CHAR(SYSDATE, 'DD-MON-YYYY');
RETURN v_date;
END get_today;
/
```

Execute / Call the Function:

```
-- Using in SQL SELECT:
SELECT get_today FROM DUAL;

-- Using in PL/SQL block:
BEGIN
DBMS_OUTPUT.PUT_LINE('Today is: ' || get_today);
END;
/
```

3.4 Function with IN Parameters

Example: Calculate the area of a rectangle.

```
CREATE OR REPLACE FUNCTION calc_area
(p_length IN NUMBER, p_width IN NUMBER)
```

```

RETURN NUMBER
IS
    v_area NUMBER;
BEGIN
    v_area := p_length * p_width;
    RETURN v_area;
END calc_area;
/

```

Execute:

```

-- From SQL:
SELECT calc_area(10, 5) AS Area FROM DUAL;

-- From PL/SQL:
DECLARE
    v_result NUMBER;
BEGIN
    v_result := calc_area(10, 5);
    DBMS_OUTPUT.PUT_LINE('Area = ' || v_result);
END;
/

```

3.5 Function Using a Database Table

Example: Return the annual salary of an employee given their employee number.

```

CREATE OR REPLACE FUNCTION annual_salary
(p_empno IN NUMBER)
RETURN NUMBER
IS
    v_annual_sal NUMBER;
BEGIN
    SELECT sal * 12
    INTO v_annual_sal
    FROM emp
    WHERE empno = p_empno;

    RETURN v_annual_sal;
EXCEPTION
    WHEN NO_DATA_FOUND THEN
        RETURN 0;
END annual_salary;
/

```

Execute:

```

-- From SQL:
SELECT ename, annual_salary(empno) AS "Annual Salary"
FROM emp;

-- From PL/SQL:
DECLARE
    v_sal NUMBER;
BEGIN
    v_sal := annual_salary(7369);
    DBMS_OUTPUT.PUT_LINE('Annual Salary: ' || v_sal);
END;
/

```

3.6 Function with Conditional Logic

Example: Return grade based on marks.

```
CREATE OR REPLACE FUNCTION get_grade
(p_marks IN NUMBER)
RETURN VARCHAR2
IS
    v_grade VARCHAR2(10);
BEGIN
    IF p_marks >= 90 THEN
        v_grade := 'A+';
    ELSIF p_marks >= 80 THEN
        v_grade := 'A';
    ELSIF p_marks >= 70 THEN
        v_grade := 'B';
    ELSIF p_marks >= 60 THEN
        v_grade := 'C';
    ELSIF p_marks >= 40 THEN
        v_grade := 'D';
    ELSE
        v_grade := 'FAIL';
    END IF;
    RETURN v_grade;
END get_grade;
/
```

Execute:

```
SELECT get_grade(85) AS Grade FROM DUAL;

BEGIN
    DBMS_OUTPUT.PUT_LINE('Grade: ' || get_grade(72));
END;
/
```

3.7 Dropping a Function

```
DROP FUNCTION function_name;

-- Example:
DROP FUNCTION get_grade;
```

3.8 Viewing Function Source Code

```
-- List all user functions:
SELECT object_name FROM user_objects WHERE object_type = 'FUNCTION';

-- View source:
SELECT text FROM user_source WHERE name = 'GET_GRADE' ORDER BY line;
```

Unit 4: Procedures vs Functions — Comparison

Aspect	Procedure	Function
Return Value	Does not return a value (uses OUT params)	Must return exactly one value

RETURN keyword	Optional (only for early exit)	Mandatory in header and body
Invocation	Called as a statement	Called within an expression
Use in SQL	Cannot be in SELECT directly	Can be used in SQL SELECT
Parameters	IN, OUT, IN OUT allowed	IN parameters only (by default)
Keyword	CREATE PROCEDURE	CREATE FUNCTION
Purpose	Perform an action	Compute and return a value
EXEC command	EXECUTE proc_name(args);	Cannot use EXECUTE directly

Unit 5: Exception Handling in Subprograms

5.1 Why Exception Handling?

Runtime errors (such as no data found, division by zero, or invalid values) can crash a PL/SQL program. Exception handling allows the program to gracefully manage these errors.

5.2 Common Pre-defined Exceptions

Exception Name	When it Occurs
NO_DATA_FOUND	SELECT INTO returns no rows
TOO_MANY_ROWS	SELECT INTO returns more than one row
ZERO_DIVIDE	Division by zero attempted
VALUE_ERROR	Arithmetic, conversion, or size error
INVALID_NUMBER	Conversion of string to number fails
DUP_VAL_ON_INDEX	Duplicate value in unique index
OTHERS	Catches any unhandled exception

5.3 Procedure with Full Exception Handling

```

CREATE OR REPLACE PROCEDURE safe_display
(p_empno IN NUMBER)
IS
    v_ename VARCHAR2(50);
    v_sal NUMBER;
BEGIN
    SELECT ename, sal
    INTO v_ename, v_sal
    FROM emp
    WHERE empno = p_empno;

    DBMS_OUTPUT.PUT_LINE('Employee : ' || v_ename);
    DBMS_OUTPUT.PUT_LINE('Salary : ' || v_sal);
EXCEPTION

```

```

WHEN NO_DATA_FOUND THEN
    DBMS_OUTPUT.PUT_LINE('ERROR: No employee with that number.');
```

```

WHEN TOO_MANY_ROWS THEN
    DBMS_OUTPUT.PUT_LINE('ERROR: Multiple records found.');
```

```

WHEN OTHERS THEN
    DBMS_OUTPUT.PUT_LINE('ERROR: ' || SQLERRM);
```

```

END safe_display;
/
```

SQLERRM & SQLCODE

SQLERRM → Returns the error message of the most recent exception.

SQLCODE → Returns the error number of the most recent exception.

These are most useful inside the WHEN OTHERS handler.

Unit 6: Practical Programs

Program 1: Insert Record Using Procedure

```

CREATE OR REPLACE PROCEDURE insert_student
(p_rollno IN NUMBER, p_name IN VARCHAR2, p_marks IN NUMBER)
IS
BEGIN
    INSERT INTO student (rollno, sname, marks)
    VALUES (p_rollno, p_name, p_marks);

    COMMIT;
    DBMS_OUTPUT.PUT_LINE('Student inserted successfully.');
```

```

EXCEPTION
    WHEN DUP_VAL_ON_INDEX THEN
        DBMS_OUTPUT.PUT_LINE('ERROR: Roll number already exists!');
```

```

    WHEN OTHERS THEN
        ROLLBACK;
        DBMS_OUTPUT.PUT_LINE('ERROR: ' || SQLERRM);
```

```

END insert_student;
/
```

Execute:

```

EXECUTE insert_student(101, 'Rahul Kumar', 88);
EXECUTE insert_student(102, 'Priya Dash', 92);
```

Program 2: Update Salary Using Procedure

```

CREATE OR REPLACE PROCEDURE update_salary
(p_empno IN NUMBER, p_percent IN NUMBER)
IS
    v_count NUMBER;
BEGIN
    SELECT COUNT(*) INTO v_count
    FROM emp WHERE empno = p_empno;

    IF v_count = 0 THEN
        DBMS_OUTPUT.PUT_LINE('No such employee!');
```

```

    ELSE
```



```

UPDATE emp
SET   sal = sal + (sal * p_percent / 100)
WHERE empno = p_empno;

COMMIT;
DBMS_OUTPUT.PUT_LINE('Salary updated successfully. ');
END IF;
EXCEPTION
WHEN OTHERS THEN
    ROLLBACK;
    DBMS_OUTPUT.PUT_LINE('Error: ' || SQLERRM);
END update_salary;
/

```

Program 3: Factorial Function

```

CREATE OR REPLACE FUNCTION factorial
(p_num IN NUMBER)
RETURN NUMBER
IS
    v_fact NUMBER := 1;
    i      NUMBER;
BEGIN
    IF p_num < 0 THEN
        RETURN -1; -- Error indicator
    END IF;

    FOR i IN 1..p_num LOOP
        v_fact := v_fact * i;
    END LOOP;

    RETURN v_fact;
END factorial;
/

```

Execute:

```

SELECT factorial(5) AS "5!" FROM DUAL;
-- Output: 120

BEGIN
    DBMS_OUTPUT.PUT_LINE('7! = ' || factorial(7));
END;
/
-- Output: 5040

```

Program 4: Check Prime Number Function

```

CREATE OR REPLACE FUNCTION is_prime
(p_num IN NUMBER)
RETURN VARCHAR2
IS
    v_flag NUMBER := 1;
    i      NUMBER;
BEGIN
    IF p_num <= 1 THEN
        RETURN 'NOT PRIME';
    END IF;

    FOR i IN 2..FLOOR(SQRT(p_num)) LOOP

```

```

    IF MOD(p_num, i) = 0 THEN
        v_flag := 0;
        EXIT;
    END IF;
END LOOP;

IF v_flag = 1 THEN
    RETURN 'PRIME';
ELSE
    RETURN 'NOT PRIME';
END IF;
END is_prime;
/

```

Execute:

```

SELECT is_prime(17) FROM DUAL; -- PRIME
SELECT is_prime(20) FROM DUAL; -- NOT PRIME

```

Program 5: Swap Two Numbers (IN OUT)

```

CREATE OR REPLACE PROCEDURE swap_values
(p_a IN OUT NUMBER, p_b IN OUT NUMBER)
IS
    v_temp NUMBER;
BEGIN
    v_temp := p_a;
    p_a := p_b;
    p_b := v_temp;
END swap_values;
/

```

Execute:

```

DECLARE
    v_x NUMBER := 10;
    v_y NUMBER := 25;
BEGIN
    DBMS_OUTPUT.PUT_LINE('Before: X=' || v_x || ', Y=' || v_y);
    swap_values(v_x, v_y);
    DBMS_OUTPUT.PUT_LINE('After: X=' || v_x || ', Y=' || v_y);
END;
/

```

Unit 7: Quick Reference & Exam Tips

7.1 Important Commands

Command	Purpose
SET SERVEROUTPUT ON	Enable output for DBMS_OUTPUT
EXECUTE proc_name(args);	Call a stored procedure
SELECT func_name(args) FROM DUAL;	Call a function from SQL
DROP PROCEDURE name;	Delete a stored procedure

DROP FUNCTION name;	Delete a stored function
SHOW ERRORS;	Display compilation errors
DESC procedure name;	Show procedure parameters
SELECT * FROM user_objects;	List all user objects

7.2 Common Exam Questions

1. What is the difference between a procedure and a function in PL/SQL?
2. Write a PL/SQL procedure to insert a record into a table.
3. Write a PL/SQL function to return the square of a number.
4. Explain the three parameter modes: IN, OUT, IN OUT with examples.
5. What is exception handling? Explain with an example in a procedure.
6. Write a function that returns the factorial of a number.
7. How do you view the source code of a stored procedure in Oracle?
8. What is the purpose of OR REPLACE in CREATE OR REPLACE PROCEDURE?

7.3 Key Points to Remember

Must-Know Facts for Exam

Procedures use EXECUTE or BEGIN...END to call; functions are called in expressions.

Functions MUST return a value; procedures do not have to.

OUT and IN OUT parameters must be variables — not literals — at call time.

WHEN OTHERS must be the LAST exception handler in the EXCEPTION block.

Use / at the end of every PL/SQL block in SQL*Plus to execute it.

COMMIT saves DML changes; ROLLBACK undoes them — use inside procedures as needed.

The DUAL table is Oracle's dummy one-row table used to call functions from SQL.

user_objects, user_source, and user_procedures are useful data dictionary views.

7.4 Compilation Errors

If a procedure or function has errors, Oracle compiles it with errors (status = INVALID). Use SHOW ERRORS to see them:

```
-- After creating a procedure/function:
SHOW ERRORS;

-- Or query the errors table:
SELECT line, position, text
FROM   user_errors
WHERE  name = 'YOUR_PROC_NAME';
```

Final Note

Practice is key! Create a sample table (like STUDENT or EMP) and write multiple procedures and functions against it.

Always test with SET SERVEROUTPUT ON enabled.

Understand when to use a procedure vs a function — this is a very common exam question.

— *End of Notes* —